# Wrong the Same Way: Universality, Not Error Rate, Governs Reward Hacking in RLVR

**Anonymous Author(s)**
Affiliation
Address
email

## Abstract

It is known that reinforcement learning with verifiable rewards (RLVR) is, at its KL-regularised optimum, a *filter*: it reweights the set of verifier-accepted responses as a whole and leaves the composition of that set exactly at the base model's, so the post-RL rate of verifier false positives (reward hacking) should simply be inherited. Recent evidence both confirms this (a preregistered contrast finds inheritance, not learned exploitation) and contradicts it (RL amplifies hacking; systematic verifier errors cause collapse). We show the two are reconciled by a quantity that is not the verifier's error rate. Writing the exact drift of $\log \pi_\theta(y \mid x)$ off the filter baseline as a sum, over all prompts, of the covariance between reward and the empirical tangent kernel isolates the one channel through which universality can act: the cross-prompt term, which vanishes *identically* for a policy whose parameters are not shared across prompts. Under an idealised kernel its sign is that of $u - \bar{V}$, where $u$ is the leak's *universality* (the fraction of prompts on which the same wrong response is accepted) and $\bar{V}$ the policy's own verifier pass rate: sharing punishes a leak on every prompt that does not accept it. We test this in a fully enumerable RLVR environment where the entire response distribution is computed exactly and the verifier's static false-positive rate is held fixed by construction. Sweeping universality at constant false-positive rate moves the hack fraction of a shared-parameter policy from 0.019 at $u = 0.016$ to $0.200 \pm 0.097$ at $u = 1$ (base model 0.126); under plain SGD the same sweep runs 0.056 to 1.000 with true accuracy falling 0.924 to 0.000, while the identically initialised per-prompt policy moves only 0.052 to 0.054. A crossed factorial attributes 59% of the variance in the log-odds drift to universality and 11% to a static false-positive rate varied eightfold. Practically, verifier hardening should be budgeted against universality: patching one universal leak and patching every idiosyncratic leak improve the measured false-positive rate identically but differ by $40 \pm 16$ points of true accuracy.

## 1 Introduction

RLVR trains a language model against a programmatic verifier: a response is rewarded if a checker accepts it [Shao et al., 2024]. Verifiers are imperfect. Rule-based matchers accept degenerate strings [Zhao et al., 2025]; test suites accept programs that pass the visible tests and nothing else [Khalifa et al., 2026]; outcome-only checkers reward a correct final answer reached by invalid reasoning [Yuan et al., 2026]. Every such false positive is a response the policy can be paid for without being right.

How much does this matter? The literature gives two incompatible answers. On one side, RLVR looks remarkably robust to verifier error: flipping a large fraction of rewards barely moves final performance [Plesner et al., 2026], noise slows learning without changing its destination [Rad et al., 2026, Wang et al., 2020], and a preregistered leaky-versus-hardened test-suite contrast finds the wrong programs

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

a trained model emits were already being emitted by the base model: "selection of pre-existing error modes rather than learned exploitation" [Zhang, 2026]. On the other, verifier exploitation is a first-order practical problem: RL amplifies hacking seeded by a trace of contamination [Khalifa et al., 2026], systematic verification error produces delay, plateau or collapse [Egashira et al., 2026], RLVR-trained models learn to game extensional verifiers [Helff et al., 2026], and production reward hacking generalises into broader misalignment [MacDiarmid et al., 2025].

There is a clean reason to expect the first answer. At the KL-regularised optimum the policy is $\pi^\star(y \mid x) \propto \pi_{\mathrm{ref}}(y \mid x)\, e^{r(x,y)/\beta}$, so within a reward level set the odds are exactly the reference model's [Korbak et al., 2022]: every accepted response, correct or exploiting, is multiplied by the same factor. RLVR is a filter on the accepted set, not a sculptor inside it, a fact noted repeatedly in the RLVR diversity literature [Gai et al., 2025, Kruszewski et al., 2026, Dymetman, 2026] and the reason RLVR sharpens rather than expands what a base model can do [Yue et al., 2025]. Applied to verifier false positives it says the post-RL hack rate is the base model's, whatever the algorithm does. Real training is not run at that optimum, and this paper asks what governs the departure.

**Contributions.**

- We write the exact instantaneous drift of $\log \pi_\theta(y \mid x)$ under RLVR gradient flow as a sum over *all* prompts of the covariance between reward and the empirical tangent kernel (Lemma 1). For a policy whose parameters are not shared across prompts, this collapses to a single term and the dynamics provably cannot depend on how many prompts share a leak (Corollary 1).
- For a shared-parameter policy we obtain a threshold (Proposition 2): the leading-order drift of a leak accepted on a fraction $u$ of prompts has the sign of $u - \bar{V}$, where $\bar{V}$ is the policy's current verifier pass rate, an observable quantity requiring no ground truth. Below the threshold, cross-prompt sharing actively suppresses the leak, because on the $1 - u$ prompts where it is *not* accepted it receives negative advantage. Neither side of the threshold involves the verifier's error rate, and neither depends on the number of prompts at fixed $u$.
- We build an RLVR environment in which the full response distribution is enumerated exactly, so every reported quantity is the true value under the policy rather than a sample estimate, and in which universality and static false-positive rate are independent knobs by construction. Matched tabular and transformer policies start from the *identical* distribution and differ only in parameter sharing.
- Empirically: the shared-parameter policy's hack fraction is monotone in universality and crosses from suppression to amplification, while the per-prompt policy is flat in $u$ (though not pinned to the filter baseline); a crossed factorial confirms the effect tracks universality and not false-positive rate; the coherence the analysis assumes is measured rather than posited; the result survives randomising which prompts share a leak, sampled GRPO, and both optimisers; and a fixed verifier-hardening budget buys far more true accuracy spent on universality than on error count.

## 2 Setup: RLVR with an exploitable verifier

Let $\mathcal{X}$ be a finite prompt set, $|\mathcal{X}| = N$, with the uniform distribution, and $\mathcal{Y}$ a finite response space. A response is *correct* if $c(x, y) = 1$; we write $g_x$ for the correct response, unique in our setting. The deployed verifier returns $r(x, y) \in \{0, 1\}$ with $r \geq c$ (no false negatives); write $\mathcal{A}(x) = \{y : r(x, y) = 1\}$ for the accepted set and $\mathcal{H}(x) = \{y : r(x, y) = 1,\, c(x, y) = 0\}$ for its false-positive part, the *leaks*. RLVR maximises $J(\theta) = \mathbb{E}_x\, \mathbb{E}_{y \sim \pi_\theta(\cdot|x)}[r(x, y)]$.

**Definition 1** (Hack fraction)**.** $\rho(\pi) = \frac{1}{N} \sum_x \pi(\mathcal{H}(x) \mid x) \,/\, \pi(\mathcal{A}(x) \mid x)$, the probability that a rewarded response is a false positive, averaged over prompts. We also track the prompt-averaged log-odds $\Lambda(\pi) = \frac{1}{N} \sum_x \log \left[\pi(\mathcal{H}(x) \mid x) \,/\, \pi(g_x \mid x)\right]$.

**Definition 2** (Leak universality)**.** For a response $h$, let $m(h) = |\{x : h \in \mathcal{H}(x)\}|$ and $u(h) = m(h)/N$. A leak with $u = 1$ is accepted everywhere; a leak with $u = 1/N$ is idiosyncratic to one prompt.

Universality is orthogonal to the verifier's error rate. A verifier that accepts exactly one wrong response per prompt has the same false-positive rate, on an audit that samples (prompt, wrong-

response) pairs as verifier benchmarks do, whether that response is the same string everywhere or a different string for each prompt.

### 2.1 The filter baseline

**Proposition 1** (RLVR is a filter; known)**.** *Assume the verifier has no false negatives, $r \geq c$, so correct responses and leaks lie in the same reward level set. Let $\pi^\star_\beta = \arg\max_\pi J(\pi) - \beta\,\mathbb{E}_x\mathrm{KL}(\pi(\cdot \mid x)\|\pi_{\mathrm{ref}}(\cdot \mid x))$. Then $\pi^\star_\beta(y \mid x) \propto \pi_{\mathrm{ref}}(y \mid x)\, e^{r(x,y)/\beta}$, so for any $y, y'$ with $r(x,y) = r(x,y')$ the odds are unchanged, and consequently $\rho(\pi^\star_\beta) = \rho(\pi_{\mathrm{ref}})$ and $\Lambda(\pi^\star_\beta) = \Lambda(\pi_{\mathrm{ref}})$ for every $\beta > 0$. The same holds at every step of any multiplicative update $\pi_{t+1}(\cdot \mid x) \propto \pi_t(\cdot \mid x)\exp(\eta f(r(x,\cdot)))$.*

The closed form is the Gibbs variational principle and is standard [Korbak et al., 2022, Kruszewski et al., 2026, Dymetman, 2026]; we state it because it is the baseline everything else is measured against, and because its consequence for reward hacking, that the composition of the accepted set is frozen at the base model's, is the prediction we are testing. As $\beta \to 0$, $\pi^\star_\beta \to \pi_{\mathrm{ref}}(\cdot \mid \mathcal{A}(x))$: the whole regularisation path is the reference model conditioned on acceptance, which is why nothing inside the accepted set can move. We verify the invariance numerically to floating-point precision (Section 5).

**Why the per-prompt statistic.** The pooled quantity $\sum_x \pi(\mathcal{H}(x) \mid x)/\sum_x \pi(\mathcal{A}(x) \mid x)$ is the $\pi(\mathcal{A}(x) \mid x)$-weighted average of the invariant per-prompt ratios, and RLVR raises those weights toward 1 at different rates. Its total excursion along the filtering path is exactly $-\operatorname{Cov}_x\big(\pi_{\mathrm{ref}}(\mathcal{A}(x) \mid x),\, \rho_x\big) \,/\, \mathbb{E}_x[\pi_{\mathrm{ref}}(\mathcal{A}(x) \mid x)]$ (Appendix A), which is *positive* whenever pass rate and hack fraction are negatively correlated across prompts. That is the usual case, since prompts a model can solve are prompts on which it needs the leak less. A practitioner watching pooled $P(\text{hack} \mid \text{rewarded})$ therefore sees it climb under pure filtering, with nothing being exploited. In our environment it rises by +0.057 under exact mirror descent while $\rho$ and $\Lambda$ do not move at all. We report the per-prompt statistics throughout.

## 3 What moves a policy off the filter baseline

Real RLVR runs first-order updates on a shared-parameter model with a small or absent KL term. We characterise the resulting drift exactly.

**Lemma 1** (Drift decomposition)**.** *Under exact-gradient flow $\dot\theta = \nabla_\theta J(\theta)$,*

$$\frac{d}{dt}\log \pi_\theta(y \mid x) = \frac{1}{N}\sum_{x' \in \mathcal{X}} \operatorname{Cov}_{y' \sim \pi_\theta(\cdot|x')}\Big[\, r(x', y'),\; K_\theta\big((x,y),(x',y')\big)\Big], \tag{1}$$

*where $K_\theta((x,y),(x',y')) = \langle \nabla_\theta \log \pi_\theta(y \mid x), \nabla_\theta \log \pi_\theta(y' \mid x')\rangle$ is the empirical tangent kernel of the log-policy.*

*Proof.* By the score-function identity $\nabla_\theta J = \frac{1}{N}\sum_{x'} \mathbb{E}_{y'}[r(x',y')\nabla_\theta \log \pi_\theta(y' \mid x')]$, and $\mathbb{E}_{y'}[\nabla_\theta \log \pi_\theta(y' \mid x')] = 0$, so the inner expectation is the (componentwise) covariance of the scalar $r$ with the vector $\nabla_\theta \log \pi_\theta$. Taking the inner product with the fixed vector $\nabla_\theta \log \pi_\theta(y \mid x)$ and using linearity of covariance in its second argument gives (1). □

Equation (1) says the log-probability of a response on *one* prompt is moved by what happens on *every* prompt, in proportion to how strongly reward covaries with tangent similarity there. Two remarks. Equation (1) is an instantaneous identity and we use it as such. And our experiments use Adam, not gradient flow: under any positive-definite preconditioner $P$, (1) holds verbatim with $K_\theta$ replaced by $K^P_\theta((x,y),(x',y')) = \langle \nabla_\theta \log \pi_\theta(y \mid x),\, P\,\nabla_\theta \log \pi_\theta(y' \mid x')\rangle$, which is still zero whenever the two gradients have disjoint support, so everything below survives. Our kernel is the finite-width empirical one, not the constant infinite-width limit of Jacot et al. [2018].

**Corollary 1** (Per-prompt parameters cannot see universality)**.** *If $\theta = (\theta_x)_{x \in \mathcal{X}}$ with $\pi_\theta(\cdot \mid x)$ a function of $\theta_x$ only, then $\nabla_{\theta_{x'}} \log \pi_\theta(y \mid x) = 0$ for $x' \neq x$, so $K_\theta((x,y),(x',y')) = 0$ off the diagonal and* (1) *reduces to the $x' = x$ term. Indeed $\nabla_{\theta_x} J = \frac{1}{N}\sum_y \pi_\theta(y \mid x)(r(x,y) -$*

$V(x))\nabla_{\theta_x} \log \pi_\theta(y \mid x)$ *depends on* $\theta_x$ *alone, so the trajectory of block* $x$ *is an autonomous ODE: a per-prompt policy generates no* $u$*-dependence of its own, and can depend on* $m$ *only through what it inherits at initialisation.*

This is what makes the tabular arm a control rather than merely a baseline. (The argument survives Adam, whose state is per-coordinate, and its momentum, which only takes linear combinations of gradients that are already block-supported; it would not survive a global gradient-norm clip, which we do not use.) Note carefully what the corollary does *not* say. It does not say the per-prompt policy sits on the filter baseline. It does not: for a softmax policy and two equally rewarded responses, exact gradient flow gives $\frac{d}{dt} \log \frac{\pi_1}{\pi_2} = (\pi_1 - \pi_2)(1 - J)$, so the more probable of the two grows, the known rich-get-richer behaviour of softmax policy gradient [Mei et al., 2020, Lochab et al., 2026]. That drift is prompt-local and carries no information about $u$, which is exactly what makes it a control rather than a confound.

For the shared-parameter case we need one empirical property.

**Definition 3** (Coherence)**.** A response $h$ is *coherent at cross-scale* $\kappa$ *and self-scale* $\kappa_s$ if the part of $K_\theta((x, h), (x', y'))$ that depends jointly on $h$ and $y'$ is $\approx \kappa\, \mathbf{1}[y' = h]$ for $x' \neq x$ and $\approx \kappa_s\, \mathbf{1}[y' = h]$ for $x' = x$; parts of the kernel that do not depend on $y'$ or on $h$ drop out (Appendix A). Coherence is a property of *how a response is computed*, not of string identity: a leak emitted by prompt-independent machinery is coherent, whereas correct answers that happen to coincide across prompts need not be. We measure $\kappa/\kappa_s$ directly (Section 5).

**Proposition 2** (Universality threshold)**.** *Let* $h$ *be coherent at scales* $(\kappa, \kappa_s)$*,* $\kappa > 0$*, accepted on* $U \subseteq \mathcal{X}$ *with* $|U| = m$*,* $u = m/N$*, and let* $g_x$ *be the correct answer at* $x$*, coherent at scales* $(\kappa_g, \kappa_{s,g})$*. Write* $V(x') = \pi_\theta(\mathcal{A}(x') \mid x')$ *and* $\bar{V} = J(\theta)$*. Then for* $x \in U$*, with* $\pi_\theta(h \mid \cdot) \equiv p_h$*,* $\pi_\theta(g. \mid \cdot) \equiv p_g$ *and* $V(\cdot) \equiv \bar{V}$*,*

$$\frac{d}{dt}\Big[\log \pi_\theta(h \mid x) - \log \pi_\theta(g_x \mid x)\Big] = \kappa\, p_h\,(u - \bar{V}) - \kappa_g\, p_g\,(u_g - \bar{V}) + \frac{1 - \bar{V}}{N}\Big[p_h(\kappa_s - \kappa) - p_g(\kappa_{s,g} - \kappa_g)\Big], \tag{2}$$

*where* $u_g$ *is the universality of* $g_x$*. If the correct answer is incoherent (*$\kappa_g \approx 0$*), the leading-order sign is that of* $u - \bar{V}$*: the leak is amplified when its universality exceeds the policy's current verifier pass rate, and suppressed when it does not. The corrections are* $O(1/N)$ *and negligible when* $N\kappa/\kappa_s \gg 1$*. The idealised kernel is an approximation and we treat the threshold as a directional prediction, not a numerical boundary; Section 5 reports what we could and could not verify about it.*

Three consequences are worth stating plainly. First, **the verifier's false-positive rate does not appear.** Accepting more distinct wrong responses per prompt raises every static audit metric but leaves (2) unchanged. Second, **sharing is two-sided**: the same mechanism that amplifies a universal leak pushes an idiosyncratic one *below* the base model's rate, because it is punished on every prompt where it is not accepted, something a per-prompt policy never does. Third, **the threshold rises during training**: $\bar{V}$ is literally $J(\theta)$, and $dJ/dt = \|\nabla J\|^2 \geq 0$ under gradient flow, so leaks of intermediate universality are amplified early and suppressed later, and only $u \approx 1$ leaks survive. This is what a delay/plateau/collapse taxonomy [Egashira et al., 2026] looks like from the inside. Note also that (2) has no leading-order dependence on $N$ at fixed $u$: the controlling quantity is the *fraction* of prompts a leak covers, not their number.

# 4 Experimental design

We want an environment in which (i) the full response distribution can be computed exactly, so that no measurement is a sample estimate; (ii) universality and false-positive rate are manipulable independently; and (iii) a shared-parameter policy and a per-prompt policy can be started from the *identical* distribution.

**Task and response space.** Prompts are the $N = 64$ pairs $(a, b)$ with $a, b \in \{2, \ldots, 9\}$; the correct response is the two-digit string $ab$. The response space is all 100 two-digit strings, so $\pi_\theta(\cdot \mid x)$ can be enumerated exactly for every prompt at every step. Policies are autoregressive over the two digits.

**Verifier.** The deployed verifier accepts the correct string plus $k$ designated wrong strings per prompt, none of which is the correct answer to any prompt (so every response is unambiguously correct, leak,

or wrong). Prompts are partitioned into blocks of size $m$; all prompts in a block share the same leak strings. Varying $m$ varies universality $u = m/N$ with the static false-positive rate held *exactly* fixed at $k/(|\mathcal{Y}| - 1)$; varying $k$ varies the static false-positive rate with universality fixed.

**Base model.** A 2-layer, 64-dimensional transformer (101,706 parameters) is supervised fine-tuned on a target that mixes the correct answer, a per-prompt competence level $\lambda_x$ spread over $[0, 0.9]$, and a small contamination rate $\varepsilon = 0.03$ of leak demonstrations, in the spirit of Khalifa et al. [2026], who find a trace of contamination enough to seed hacking. Competence is assigned by a fixed permutation independent of the block structure, so difficulty is uncorrelated with universality. The base model reaches accuracy 0.440 and hack fraction 0.126. The SFT target is identical in every condition, but the realised base, itself a shared-parameter network, fits a universal leak map slightly better than an idiosyncratic one, so the measured base hack fraction still varies by 0.028 across conditions. We therefore report drifts from each run's own base.

**Matched policies.** The tabular policy has free per-prompt parameters in the same autoregressive factorisation, which parameterises the whole simplex over responses, and is initialised to reproduce the base transformer's distribution to $10^{-4}$. Both are then trained with identical objectives and learning-rate grids, differing only in parameter sharing.

**Protocol.** Unless stated otherwise we run exact-expectation policy gradient with the exact mean baseline (the expectation-limit of RLOO / Dr. GRPO [Ahmadian et al., 2024, Liu et al., 2025]; group-relative variants such as Yu et al. [2025] share this structure) for 400 steps with Adam, three to five seeds per cell (Appendix B, Table 1), and report the operating point selected by the criterion a practitioner would use: the smallest learning rate in the grid whose final *verifier* reward is within $10^{-3}$ of the best. That tie-break is conservative for our hypothesis (smaller steps drift less) and it matters, because several learning rates saturate the verifier at reward 1 with wildly different true accuracy. Section 5 also reports sampled GRPO with group-standard-deviation normalisation and plain SGD.

# 5 Results

We report drifts from each run's own base model: $\Delta\rho$ (a probability difference) and $\Delta\Lambda$ (a log-odds difference), both of which Proposition 1 makes exactly zero under the filter baseline and both of which are therefore immune to the residual base-model mismatch described in Section 4. Absolute $\rho$ is plotted with the base curve.

## 5.1 The filter baseline holds exactly, and gradient training leaves it

Exact mirror descent leaves $\rho$ invariant to $8.6 \times 10^{-9}$ and $\Lambda$ to $9.6 \times 10^{-8}$ over 400 steps, and the KL-regularised optimum leaves $\rho$ invariant to $8.6 \times 10^{-9}$ across $\beta \in$[0.05, 2.0], i.e. floating-point zero, as Proposition 1 requires. Over the same trajectory the *pooled* hack fraction moves by +0.057: a practitioner tracking it would infer growing exploitation from pure filtering. Lemma 1 matches finite differences to $2.9 \times 10^{-4}$ for the transformer, and the per-prompt policy's cross-prompt tangent kernel is exactly 0 against the transformer's 0.098 of its own self-kernel: Corollary 1 in numbers.

## 5.2 Universality governs the outcome; the error rate does not

Figure 1 is the main result. Holding the verifier's static false-positive rate exactly constant and sweeping only how many prompts share the accepted wrong response moves the shared-parameter policy's hack fraction from $0.019 \pm 0.006$ at $u = 0.016$ to $0.200 \pm 0.097$ at $u = 1$ (5 seeds), against a base model value of 0.126; as a drift, $\Delta\rho$ runs from $-0.110 \pm 0.005$ to $+0.069 \pm 0.097$. True accuracy falls from 0.980 to 0.799. Both directions are predicted: below the threshold, sharing suppresses the leak far below the base rate; above it, sharing manufactures hacking that the filter baseline forbids.

Three things must be said plainly about the $u = 1$ endpoint. It is *heterogeneous across seeds*, not merely noisy: the five final hack fractions are 0.02, 0.06, 0.14, 0.22, 0.56, so the mean is carried by the seeds that collapse. The amplification is also *concentrated across prompts*: at $u = 1$ a mean of 20% of prompts end with the leak more likely than the correct answer, against 2% at $u = 0.016$.

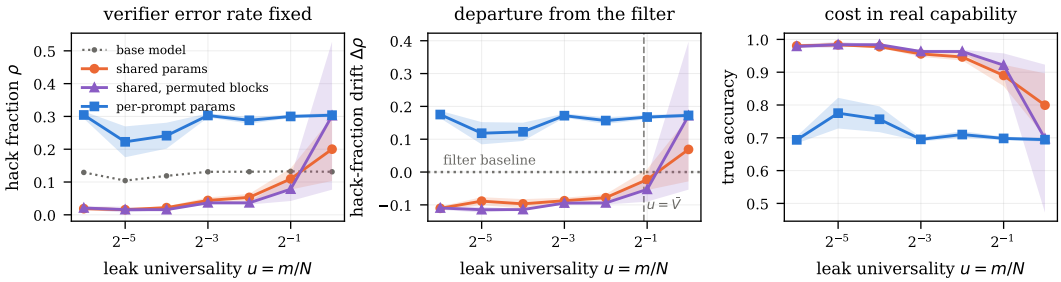



Figure 1: **Universality, at a fixed verifier error rate, decides whether RLVR suppresses or amplifies a leak.** Every verifier accepts exactly one wrong response per prompt, so the static false-positive rate is identical, and only the number of prompts sharing it varies. Left: hack fraction after training, with the base model. Centre: drift from the base model, zero under the filter baseline; dashed line is $u = \bar{V}$. Right: true accuracy. Purple repeats the sweep with randomised block membership. Bands are $\pm 1$ s.e.m. over seeds.

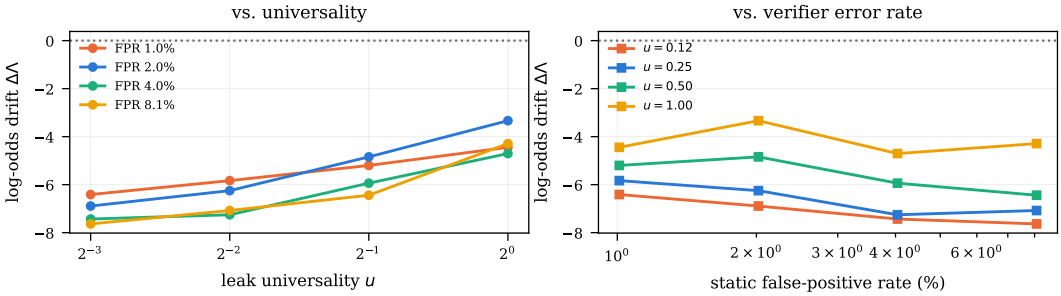



Figure 2: **The controlling quantity is universality, not the verifier's error rate.** Left: log-odds drift against universality, one line per false-positive rate. Right: the same data against the static false-positive rate, one line per universality, common $y$-scale. Varying accepted wrong responses per prompt from 1–8 moves the false-positive rate over 1.0–8.1% and mildly *reduces* hacking; universality moves the drift monotonically and much further.

That concentration is why $\Delta\rho$ crosses zero while $\Delta\Lambda$, which averages logs and is dominated by the suppressed majority, rises monotonically from -8.8 to -2.8 but stays negative: the two statistics agree on direction and disagree on "amplified overall". And the worst case is starker than the operating point: among the 21 runs at $u = 1$ reaching verifier reward $\geq 0.99$, one reaches reward 1.000 with hack fraction 1.000 and true accuracy 0.000, whereas the worst of the 10 reward-saturated runs at $u = 0.016$ still has accuracy 0.966. Finally, the sign change is bracketed by our grid between $u = 0.5$ and $u = 1$ and we do not resolve its location; it lies above the base-model $\bar{V} = 0.473$ as the theory requires, but that is a weak test. Randomising which prompts share a leak reproduces this closely (0.021 to 0.302, drift -0.110 to +0.172, crossing at $u = 0.62$, accuracy falling to 0.697), so the effect is not an artefact of contiguous blocks coinciding with a single prompt token. That design also happens to equalise the base model: base $\rho$ is 0.131 in every condition, varying by only 0.0011, against 0.028 with contiguous blocks. Its per-prompt arm is flat as well (+0.171 to +0.157, slope +0.0003 versus +0.0353 for the shared-parameter policy).

The per-prompt policy, started from the identical distribution and given the identical learning-rate grid, is flat. The cleanest statement comes from the permuted-block sweep, where the base model is equal across conditions to within 0.0011: regressing $\Delta\rho$ on $\log_2 m$ gives +0.0003 per doubling for the per-prompt policy against +0.0353 for the transformer, two orders of magnitude apart. With contiguous blocks, where the inherited base itself varies by 0.028, the same regression gives +0.0045 against +0.0246. The per-prompt arm does not sit *on* the filter baseline: its $\Delta\rho$ is +0.175 to +0.173, and under sampled GRPO it sits below the baseline instead, so we do not attribute the offset to any single cause. But its departure carries no information about universality, which is what Corollary 1 requires and what the comparison tests. The claim of this paper is about the slope, not the level.

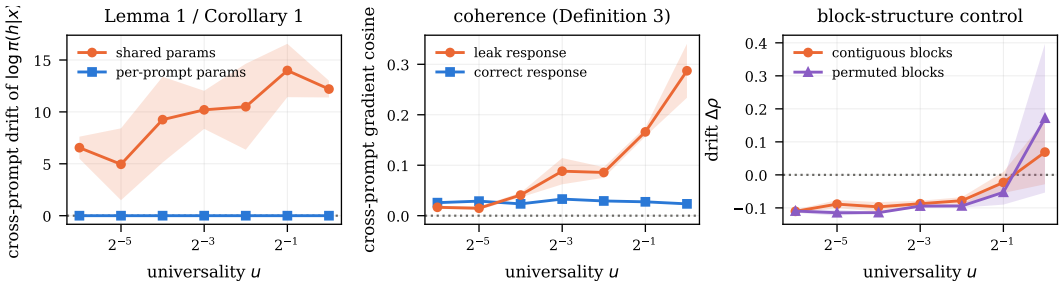



Figure 3: **Mechanism.** Left: the cross-prompt component of the exact drift of $\log \pi_\theta(h \mid x)$ at the base model (Lemma 1), identically zero for per-prompt parameters, as Corollary 1 requires. Centre: mean pairwise cosine between per-prompt gradients, i.e. the coherence of Definition 3, measured rather than assumed. Right: contiguous versus randomised block membership.

Figure 2 crosses universality against the static false-positive rate. Here the SFT target keeps its uniform component off the leak strings, so total base hack *mass* is exactly $\varepsilon$ for every $k$; the base hack *fraction* still rises mildly with $k$ (mean 0.128, spread 0.054 across cells) because diverting the uniform floor also changes the mass on the correct answer, which is one more reason to analyse drifts. We use the log-odds drift $\Delta\Lambda$ here rather than $\Delta\rho$, because it measures the multiplicative change in total leak mass and is therefore invariant to how a fixed base propensity is divided among $k$ strings. A two-way decomposition attributes 59% of its variance to universality (swept over $u \in [0.125, 1]$) and 11% to the false-positive rate (swept over an eightfold range): averaged over $k$, $\Delta\Lambda$ moves from -7.09 at $u = 0.125$ to -4.19 at $u = 1$, while averaged over $u$ it moves only from -5.47 to -6.36 as the false-positive rate grows eightfold. Note the sign of the latter: accepting *more* wrong responses per prompt does not increase hacking here, and mildly decreases it, because the base model's fixed propensity to emit an accepted-wrong response is divided among more and individually rarer strings. An audit that counts false positives is measuring something with the wrong sign as well as the wrong magnitude.

### 5.3 The mechanism is cross-prompt gradient aggregation

Figure 3 tests the mechanism directly. Lemma 1 splits the exact drift of $\log \pi_\theta(h \mid x)$ into an own-prompt term, present for any policy, and a cross-prompt term that is the entire contribution of parameter sharing. Computing both by autograd at the base model, the cross-prompt term is exactly 0 for the per-prompt policy at every universality, where Corollary 1 is exact rather than approximate, while for the shared policy it grows from $6.6 \pm 1.1$ at $u = 0.016$ to $12.2 \pm 0.8$ at $u = 1$. Coherence (Definition 3) is likewise measured, not posited: the mean pairwise cosine between per-prompt gradients of the leak rises from +0.02 to +0.29 while for correct answers it stays at +0.03, even though those are not perfectly prompt-specific either ($u_g = 0.037$).

**What we could not verify.** The $u > \bar{V}$ threshold cannot be pinned down pointwise here. At initialisation the cross-prompt term is positive at every $u$, dominated by a transient in which every accepted response's logits rise together; after 60 RL steps that transient has passed but $\bar{V}$ has already reached 0.993, so the predicted threshold sits at essentially 1 and the cross-prompt log-odds drift is small and negative everywhere ($-0.067$ to $-0.030$), consistent with the proposition for $u \leq \bar{V}$, but not a sign change. The observable window is narrow; what survives is the directional content.

### 5.4 Under GRPO, under SGD, and with a KL penalty

The effect is not an artefact of exact-expectation gradients or of Adam. Under sampled GRPO with group size 8 and group-standard-deviation normalisation, $\Delta\rho$ runs from -0.103 to +0.543, crossing zero at $u = 0.24$. The analysis also predicts that at fixed $u$ the outcome should not depend on the number of prompts. Repeating the sweep at $N = 16$ and $N = 64$ bears this out on the discriminating comparison: at $u = 1$ the two agree ($-0.004 \pm 0.084$ versus $+0.010 \pm 0.045$), whereas at the same *block size* $m = 16$, which is $u = 1$ for the small prompt set and $u = 0.25$ for the large one, they differ by -0.079 against $-0.004$. Across the 5 matched values of $u$ the largest gap is 0.065, comparable to

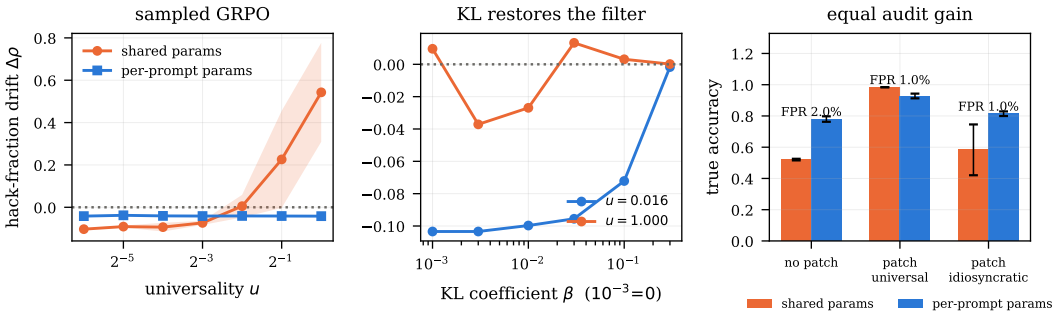



Figure 4: **Practice.** Left: sampled GRPO reproduces the pattern. Centre: a KL penalty pulls the policy back toward the filter baseline from both sides. Right: two verifier patches that improve the measured false-positive rate identically buy very different true accuracy.

the $O(1/N)$ correction the appendix flags as non-negligible at $N = 16$. The controlling quantity is the fraction of prompts a leak covers, not their number.

Plain SGD, the optimiser the analysis is written for, gives the sharpest separation in the paper: the shared-parameter policy's hack fraction runs 0.056 to 1.000 with true accuracy falling 0.924 to 0.000, crossing at $u = 0.07$ (slope +0.179 per doubling), while the per-prompt policy moves from 0.052 to 0.054, a slope of -0.001, and *below* its base throughout, exactly the prompt-local rich-get-richer drift of Appendix A. Figure 5 has the full learning-rate grid, the trajectories, and the prompt-set-size sweep.

Adding a KL penalty pulls the policy back toward the filter baseline, as Proposition 1 leads one to expect, but it does so in *both* directions, and that is the interesting part. Raising $\beta$ from 0 to 0.3 takes $\Delta\rho$ from -0.103 to -0.002 at $u = 0.016$, monotonically, and in the direction of removing the *beneficial* suppression that parameter sharing was providing, at a cost of 0.974 to 0.822 in true accuracy. At $u = 1$ the picture is within noise: $\Delta\rho$ over $\beta \in \{0, \ldots, 0.3\}$ is +0.010, -0.037, -0.027, +0.013, +0.003, +0.000, with the best accuracy at a small non-zero $\beta$ rather than at either end. These runs use a single fixed learning rate rather than the reward-selected operating point, which is why their $u = 1$ drift is smaller than in Figure 1. The defensible reading is that KL is not a free defence: it is a commitment to the base model's composition of the accepted set, which helps only when that composition is better than what training would otherwise reach.

### 5.5 Budgeting verifier hardening

We give a verifier two leak families, one universal string plus one idiosyncratic string per prompt, and compare two patches that remove exactly the same number of accepted wrong (prompt, response) pairs, and so improve any static audit identically. Removing the universal leak leaves the policy at 0.984 true accuracy; removing every idiosyncratic leak instead leaves it at 0.583, a gap of $40 \pm 16$ points (3 seeds) from patches an audit rates as equally valuable (unpatched: 0.521). The per-prompt policy, which by Corollary 1 cannot respond to universality at all, shows a gap of only 11 points between the same two verifiers. That residue is what the base model already carried, since the SFT stage is itself shared-parameter (base hack fractions 0.064 and 0.094). The remaining 29 points are produced by RL.

## 6 Related work

**RLVR as filtering.** The exponential-tilting form of the KL-regularised optimum, and the preservation of relative probabilities among equally rewarded responses that follows from it, is established [Korbak et al., 2022, Gai et al., 2025, Kruszewski et al., 2026, Dymetman, 2026, Lochab et al., 2026] and underlies the finding that RLVR sharpens rather than expands base-model capability [Yue et al., 2025]. That literature reads the invariance as a statement about *diversity* among correct answers; we read it as one about the composition of the accepted set, and use it as the null model for reward hacking. The rich-get-richer behaviour among tied responses under softmax policy gradient is likewise known [Mei et al., 2020, Lochab et al., 2026]; we use it as a control.

**Verifier error.** One line of work models verifier error as class-conditional label noise and finds RLVR robust to it [Wang et al., 2020, Rad et al., 2026, Cai et al., 2025, Plesner et al., 2026]; a related result is that even spurious rewards can improve some models [Shao et al., 2025]. Such noise is by construction independent of the response, so the policy cannot steer toward it. Egashira et al. [2026] break that assumption with content-dependent verification error and observe delay, plateau or collapse depending on the error *pattern*; Zhang [2026] finds inheritance rather than learned exploitation with naturally leaky test suites; Khalifa et al. [2026] finds amplification; Nishimura-Gasparian et al. [2026] raise the share of specification gaming attributable to generalisation as an open question. Universality is one parameter that orders these observations. Measurements of verifier false positives [Zhao et al., 2025, Huang et al., 2025, Xu et al., 2025, Yuan et al., 2026] report error rates, not universality; we argue universality is the quantity to report.

**Reward hacking theory.** Everitt et al. [2017] formalise state-dependent reward corruption with strong no-free-lunch results, Skalse et al. [2022] characterise when a proxy is unhackable, and Gao et al. [2022] give the empirical overoptimisation curves. These describe what a corrupted reward *can* do; we ask what an ordinary first-order optimiser on a shared-parameter model does with one. Work modifying the group-relative advantage estimator [Yang et al., 2026, Zeng et al., 2025] is orthogonal: our effect survives with exact-expectation gradients, where no estimator noise exists.

# 7 Limitations and conclusion

Proposition 2 assumes a behaviour-aligned tangent kernel and an approximately constant pass rate; we measure the former, and find it varies fourteenfold over the sweep, so the threshold is a first-order prediction, not a boundary, and Section 5 says where we could not verify it. The environment is small by design: a two-token response space is what makes exact enumeration possible, and the leak is a designated string rather than a strategy. We study one leak family per condition where real verifiers leak through many; our seed counts are three to five with a heterogeneous $u = 1$ outcome; and our policies start from an SFT base with injected contamination [Khalifa et al., 2026], so leaks a model finds on its own may differ in universality. A stronger control than ours, masking the cross-prompt gradient contribution inside one shared-parameter model, would isolate sharing without changing architecture.

RLVR is known to be a filter at its KL-regularised optimum (Proposition 1). What breaks that guarantee is not how often a verifier is wrong but whether it is wrong in the same way across prompts: parameter sharing lets a leak collect gradient from every prompt that accepts it while every prompt that does not accept it pushes back, and without sharing that channel is identically zero. Universality, not error rate, is therefore the quantity to measure, and the quantity against which verifier hardening should be budgeted.

# References


Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting REINFORCE style optimization for learning from human feedback in LLMs. *arXiv preprint arXiv:2402.14740*, 2024.

Xin-Qiang Cai, Wei Wang, Feng Liu, Tongliang Liu, Gang Niu, and Masashi Sugiyama. Reinforcement learning with verifiable yet noisy rewards under imperfect verifiers. *arXiv preprint arXiv:2510.00915*, 2025.

Marc Dymetman. Binary rewards and reinforcement learning: Fundamental challenges. *arXiv preprint arXiv:2605.02375*, 2026.

Kazuki Egashira, Mark Vero, Jasper Dekoninck, Florian E. Dorner, Robin Staab, and Martin Vechev. Delay, plateau, or collapse: Evaluating the impact of systematic verification error on RLVR. In *Conference on Language Modeling (COLM)*, 2026.

Tom Everitt, Victoria Krakovna, Laurent Orseau, Marcus Hutter, and Shane Legg. Reinforcement learning with a corrupted reward channel. In *International Joint Conference on Artificial Intelligence (IJCAI)*, 2017.

Jingchu Gai, Guanning Zeng, Huaqing Zhang, and Aditi Raghunathan. Differential smoothing mitigates sharpening and improves LLM reasoning. *arXiv preprint arXiv:2511.19942*, 2025.

Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. *arXiv preprint arXiv:2210.10760*, 2022.

Lukas Helff, Quentin Delfosse, David Steinmann, Ruben Härle, Hikaru Shindo, Patrick Schramowski, Wolfgang Stammer, Kristian Kersting, and Felix Friedrich. LLMs gaming verifiers: RLVR can lead to reward hacking. *arXiv preprint arXiv:2604.15149*, 2026.

Yuzhen Huang, Weihao Zeng, Xingshan Zeng, Qi Zhu, and Junxian He. From accuracy to robustness: A study of rule- and model-based verifiers in mathematical reasoning. *arXiv preprint arXiv:2505.22203*, 2025.

Arthur Jacot, Franck Gabriel, and Clément Hongler. Neural tangent kernel: Convergence and generalization in neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

Muhammad Khalifa, Zohaib Khan, Omer Tafveez, Hao Peng, and Lu Wang. Countdown-code: A testbed for studying the emergence and generalization of reward hacking in RLVR. *arXiv preprint arXiv:2603.07084*, 2026.

Tomasz Korbak, Ethan Perez, and Christopher L Buckley. RL with KL penalties is better viewed as Bayesian inference. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, 2022.

Germán Kruszewski, Pierre Erbacher, Jos Rozen, and Marc Dymetman. Whatever remains must be true: Filtering drives reasoning in LLMs, shaping diversity. In *International Conference on Learning Representations (ICLR)*, 2026.

Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding R1-zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025.

Anamika Lochab, Bolian Li, and Ruqi Zhang. Uniform-correct policy optimization: Breaking RLVR's indifference to diversity. *arXiv preprint arXiv:2605.00365*, 2026.

Monte MacDiarmid, Benjamin Wright, Jonathan Uesato, Joe Benton, Jon Kutasov, Sara Price, Naia Bouscal, Sam Bowman, Trenton Bricken, Alex Cloud, Carson Denison, Johannes Gasteiger, Ryan Greenblatt, Jan Leike, Jack Lindsey, Vlad Mikulik, Ethan Perez, Alex Rodrigues, Drake Thomas, Albert Webson, Daniel Ziegler, and Evan Hubinger. Natural emergent misalignment from reward hacking in production RL. *arXiv preprint arXiv:2511.18397*, 2025.

Jincheng Mei, Chenjun Xiao, Csaba Szepesvári, and Dale Schuurmans. On the global convergence rates of softmax policy gradient methods. In *International Conference on Machine Learning (ICML)*, 2020.

Kei Nishimura-Gasparian, Robert McCarthy, and David Lindner. Towards understanding specification gaming in reasoning models. *arXiv preprint arXiv:2605.02269*, 2026.

Andreas Plesner, Francisco Guzmán, and Anish Athalye. An imperfect verifier is good enough: Learning with noisy rewards. *arXiv preprint arXiv:2604.07666*, 2026.

Ali Rad, Khashayar Filom, Darioush Keivan, Peyman Mohajerin Esfahani, and Ehsan Kamalinejad. Rate or fate? RLV$^{\varepsilon}$R: Reinforcement learning with verifiable noisy rewards. *arXiv preprint arXiv:2601.04411*, 2026.

Rulin Shao, Shuyue Stella Li, Rui Xin, Scott Geng, Yiping Wang, Sewoong Oh, Simon Shaolei Du, Nathan Lambert, Sewon Min, Ranjay Krishna, Yulia Tsvetkov, Hannaneh Hajishirzi, Pang Wei Koh, and Luke Zettlemoyer. Spurious rewards: Rethinking training signals in RLVR. *arXiv preprint arXiv:2506.10947*, 2025.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward hacking. *arXiv preprint arXiv:2209.13085*, 2022.

Jingkang Wang, Yang Liu, and Bo Li. Reinforcement learning with perturbed rewards. In *AAAI Conference on Artificial Intelligence*, 2020.

Zhangchen Xu, Yuetai Li, Fengqing Jiang, Bhaskar Ramasubramanian, Luyao Niu, Bill Yuchen Lin, and Radha Poovendran. TinyV: Reducing false negatives in verification improves RL for LLM reasoning. *arXiv preprint arXiv:2505.14625*, 2025.

Fengkai Yang, Zherui Chen, Xiaohan Wang, Xiaodong Lu, Jiajun Chai, Guojun Yin, Wei Lin, Shuai Ma, Fuzhen Zhuang, Deqing Wang, Yaodong Yang, Jianxin Li, and Yikun Ban. Your group-relative advantage is biased. *arXiv preprint arXiv:2601.08521*, 2026.

Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao, Yonghui Wu, and Mingxuan Wang. DAPO: An open-source LLM reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.

Youliang Yuan, Qiuyang Mang, Jingbang Chen, Hong Wan, Xiaoyuan Liu, Junjielong Xu, Jen-tse Huang, Wenxuan Wang, Wenxiang Jiao, and Pinjia He. Curing miracle steps in LLM mathematical reasoning with rubric rewards. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2026.

Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

Guanning Zeng, Zhaoyi Zhou, Daman Arora, and Andrea Zanette. Shrinking the variance: Shrinkage baselines for reinforcement learning with verifiable rewards. *arXiv preprint arXiv:2511.03710*, 2025.

Chuyifei Zhang. When the reward suite is leaky: A preregistered causal contrast of natural verifier false positives in RLVR. *arXiv preprint arXiv:2607.11022*, 2026.

Yulai Zhao, Haolin Liu, Dian Yu, Sunyuan Kung, Meijia Chen, Haitao Mi, and Dong Yu. One token to fool LLM-as-a-judge. *arXiv preprint arXiv:2507.08794*, 2025.

# A Proofs

## A.1 Proposition 1

Fix $x$. The objective $\sum_y \pi(y) r(x,y) - \beta \sum_y \pi(y) \log \frac{\pi(y)}{\pi_{\mathrm{ref}}(y)}$ over the simplex is strictly concave for $\beta > 0$; setting the derivative of the Lagrangian to zero gives $\pi^\star_\beta(y \mid x) = \pi_{\mathrm{ref}}(y \mid x) e^{r(x,y)/\beta} / Z_\beta(x)$ with $Z_\beta(x) = \sum_{y'} \pi_{\mathrm{ref}}(y' \mid x) e^{r(x,y')/\beta}$. For $y, y'$ with $r(x,y) = r(x,y')$ the normaliser and the exponential factor cancel in the ratio. Since $r \geq c$ we have $\mathcal{H}(x) \subseteq \mathcal{A}(x)$ and $r \equiv 1$ on $\mathcal{A}(x)$, so

$$\frac{\pi^\star_\beta(\mathcal{H}(x) \mid x)}{\pi^\star_\beta(\mathcal{A}(x) \mid x)} = \frac{e^{1/\beta} \pi_{\mathrm{ref}}(\mathcal{H}(x) \mid x) / Z_\beta(x)}{e^{1/\beta} \pi_{\mathrm{ref}}(\mathcal{A}(x) \mid x) / Z_\beta(x)} = \frac{\pi_{\mathrm{ref}}(\mathcal{H}(x) \mid x)}{\pi_{\mathrm{ref}}(\mathcal{A}(x) \mid x)}$$

for every $x$, hence $\rho$ and $\Lambda$ (the latter whenever $\pi(\mathcal{H}(x) \mid x) > 0$) are unchanged. For a multiplicative update $\pi_{t+1}(y \mid x) \propto \pi_t(y \mid x) e^{\eta f(r(x,y))}$ the same cancellation applies at every step.

**Excursion of the pooled statistic.** Write $a_x = \pi_{\mathrm{ref}}(\mathcal{A}(x) \mid x)$, $\rho_x = \pi_{\mathrm{ref}}(\mathcal{H}(x) \mid x)/a_x$ and $s = e^{1/\beta}$. Then $\pi^\star_\beta(\mathcal{A}(x) \mid x) = s\,a_x/(1 - a_x + s\,a_x)$, and the pooled statistic is $P(s) = \sum_x \rho_x w_x(s)/\sum_x w_x(s)$ with $w_x(s) = sa_x/(1 - a_x + sa_x)$. At $s = 1$ (no RL) $P = \sum_x a_x\rho_x/\sum_x a_x$; as $s \to \infty$ every $w_x \to 1$ and $P \to \frac{1}{N}\sum_x \rho_x = \rho(\pi_{\mathrm{ref}})$. The total excursion is therefore

$$P(\infty) - P(1) \;=\; \frac{1}{N}\sum_x \rho_x \;-\; \frac{\sum_x a_x\rho_x}{\sum_x a_x} \;=\; -\,\frac{\mathrm{Cov}_x(a_x, \rho_x)}{\mathbb{E}_x[a_x]},$$

which is positive whenever pass rate and hack fraction are negatively correlated across prompts. So the pooled statistic drifts upward under pure filtering and is only invariant in the degenerate case $\rho_x \equiv$ const; it is, however, asymptotically *unbiased* for the invariant $\rho$, since it converges to it in the strong-RL limit.

### A.2 Lemma 1

$\nabla_\theta J = \frac{1}{N}\sum_{x'}\sum_{y'} \pi_\theta(y' \mid x')r(x', y')\nabla_\theta \log \pi_\theta(y' \mid x') = \frac{1}{N}\sum_{x'} \mathbb{E}_{y'}[r(x', y')\nabla_\theta \log \pi_\theta(y' \mid x')]$. Since $\mathbb{E}_{y'}[\nabla_\theta \log \pi_\theta(y' \mid x')] = 0$, the expectation equals $\mathrm{Cov}_{y'}(r, \nabla_\theta \log \pi_\theta)$, understood componentwise. For a fixed vector $v = \nabla_\theta \log \pi_\theta(y \mid x)$, $\langle v, \mathrm{Cov}(r, g)\rangle = \mathbb{E}[r\langle v, g\rangle] - \mathbb{E}[r]\mathbb{E}[\langle v, g\rangle] = \mathrm{Cov}(r, \langle v, g\rangle)$, so the inner product passes inside. □

### A.3 Corollary 1

With $\theta = (\theta_x)$ and $\pi_\theta(\cdot \mid x)$ a function of $\theta_x$ only, $\nabla_{\theta_{x'}} \log \pi_\theta(y \mid x) = 0$ for $x' \neq x$, so every off-diagonal kernel entry is a sum of products with a zero factor. Moreover $\nabla_{\theta_x} J = \frac{1}{N}\sum_y \pi_\theta(y \mid x)(r(x, y) - V(x))\nabla_{\theta_x} \log \pi_\theta(y \mid x)$ involves only $\theta_x$, so the flow restricted to block $x$ is autonomous and depends on the verifier only through $r(x, \cdot)$. □

**Rich-get-richer under vanilla policy gradient.** For a single prompt with free logits $\theta_y$ and $J = \sum_y \pi_y r_y$, $\partial J/\partial\theta_z = \pi_z(r_z - J)$, so for two responses with $r_1 = r_2 = 1$, $\frac{d}{dt}\log(\pi_1/\pi_2) = \dot\theta_1 - \dot\theta_2 = (\pi_1 - \pi_2)(1 - J)$. The odds between equally rewarded responses are therefore *not* preserved by vanilla policy gradient, even with no parameter sharing; they move toward whichever response already has more mass, at a rate that vanishes only as $J \to 1$. This is the prompt-local drift referred to in Section 3; it is independent of $u$ by Corollary 1.

### A.4 Proposition 2

Two parts of the kernel drop out before Definition 3 is applied: any component of $K_\theta((x, h), (x', y'))$ that does not depend on $y'$ contributes zero covariance in (1), and any component that does not depend on $h$ is common to the leak and to the correct answer and cancels in the within-prompt log-odds. Substituting the definition, the covariance at $x'$ is

$$\mathrm{Cov}_{y'}[r(x', y'), K_\theta((x, h), (x', y'))] = \kappa\Big(\pi_\theta(h \mid x')r(x', h) - \pi_\theta(h \mid x')V(x')\Big) = \kappa\,\pi_\theta(h \mid x')\big(r(x', h) - V(x')\big),$$

using $\mathbb{E}_{y'}[r] = V(x')$, with $\kappa$ replaced by $\kappa_s$ in the $x' = x$ term. Since $r(x', h) = \mathbf{1}[x' \in U]$, for $x \in U$,

$$\frac{d}{dt}\log \pi_\theta(h \mid x) = \frac{\kappa}{N}\Big[\sum_{x' \in U\setminus\{x\}} \pi_\theta(h \mid x')(1 - V(x')) - \sum_{x' \notin U} \pi_\theta(h \mid x')V(x')\Big] + \frac{\kappa_s}{N}\pi_\theta(h \mid x)(1 - V(x)).$$

With $\pi_\theta(h \mid \cdot) \equiv p_h$ and $V \equiv \bar V$ the bracket is $p_h[(m - 1)(1 - \bar V) - (N - m)\bar V] = p_h[N(u - \bar V) - (1 - \bar V)]$, giving $\kappa p_h(u - \bar V) + \frac{p_h(1-\bar V)}{N}(\kappa_s - \kappa)$. The identical computation for $g_x$ with scales $(\kappa_g, \kappa_{s,g})$ and universality $u_g$ gives $\kappa_g p_g(u_g - \bar V) + \frac{p_g(1-\bar V)}{N}(\kappa_{s,g} - \kappa_g)$; subtracting yields (2). □

**Validity of the leading-order threshold.** The $O(1/N)$ terms are negligible when $N\kappa/\kappa_s \gg 1$. We measure $\kappa/\kappa_s$ as the mean cross-prompt gradient cosine for the leak, and it is not constant: +0.29 at $u = 1$ gives $N\kappa/\kappa_s \approx 18$, but +0.02 at $u = 0.016$ gives $N\kappa/\kappa_s \approx 1.3$, at which point

the correction is of the same order as the leading term. The leading-order threshold is therefore an argument about the high-universality end only; the suppression we observe at low $u$ is an empirical finding, not a prediction of (2). Proposition 2 also treats $\kappa$ as a constant while we measure it growing roughly fourteenfold with $u$, so the observed growth of the cross-prompt drift is jointly attributable to $\kappa(u)$ and to $(u - \bar{V})$ and the proposition does not separate them. Both are reasons we do not read a numerical boundary off the experiments.

**Reading the threshold.** $\bar{V}$ is the policy's *verifier* pass rate, observable during training without any ground truth. A leak is amplified when it is accepted more often than the policy currently passes. The statement is directional and first-order: it identifies the sign of the leading-order drift and the quantity controlling it, not an exact boundary in a model whose kernel is only approximately coherent.

# B Environment, model and training details

**Task.** Prompts are all $(a, b)$ with $a, b \in \{2, \ldots, 9\}$ ($N = 64$); the correct response is the two-digit decimal string of $ab$ (all products lie in $[4, 81]$). The response space is all 100 two-digit strings. Reducing the operand set to $\{2, \ldots, 5\}$ gives $N = 16$ for the prompt-set-size sweep, which uses three seeds and the learning-rate grid $\{3 \times 10^{-4}, 10^{-3}, 3 \times 10^{-3}\}$ shared by both arms. Note the correct answers are not perfectly prompt-specific ($2 \times 6$ and $3 \times 4$ both give 12), so they carry a universality of their own, $u_g = 0.037$. Under Definition 3 what matters is measured coherence rather than string identity, and we find the correct responses to be near-incoherent across prompts (+0.03) while the leak is not; but $u_g$ is small and non-zero, and we report it rather than assume it away.

**Leak assignment.** Leak strings are drawn without replacement from the 69 two-digit strings that are not the correct answer of any prompt, so a leak is never a correct answer anywhere and every response is unambiguously classified. Prompts are partitioned into $N/m$ blocks; each block receives $k$ leak strings shared by all its prompts. The verifier's static false-positive rate is exactly $k/99$ in every condition, independent of $m$.

Blocks are contiguous in the default configuration, and with prompts enumerated first-operand-major a contiguous block of size $m \geq 8$ is exactly "first operand $= a$", a property the transformer can read off a single input token. That would confound universality with how simple the prompt-to-leak map is to represent, so we run the whole sweep a second time with block membership given by a random permutation of prompts and leak strings drawn independently per $(m, k, \text{seed})$ rather than nested across $m$. Both versions appear in every figure.

**What the base model does and does not control.** The SFT target puts identical hack mass on every prompt in every condition, but the realised base model is a shared-weight transformer that fits a universal leak map slightly better than an idiosyncratic one, so the measured base hack fraction still varies by 0.028 across universality conditions. We therefore report *drifts* from each run's own base model, $\Delta\rho$ and $\Delta\Lambda$, which Proposition 1 makes exactly zero under the filter baseline and which are immune to that residual mismatch. Absolute $\rho$ is shown alongside the base curve so both are visible.

In the universality×error-rate factorial an additional correction is needed. The uniform component of the SFT target would otherwise place mass on each of the $k$ leak strings, so base hack mass would grow with $k$ and the two factors would not be separable. For that experiment the uniform component is spread only over responses that are neither the correct answer nor a leak, which makes total base hack mass exactly $\varepsilon$ for every $k$: measured base $\rho = 0.128$ in every cell. This is the one place where the SFT target differs between experiments, and it is the version the factorial requires.

**Base model.** A decoder-only transformer with 2 layers, model dimension 64, 4 heads and a 10-way output head over digits (101,706 parameters) is trained for 600 Adam steps at $3 \times 10^{-3}$ on the exact target $q(y \mid x) = (1 - \varepsilon)[\lambda_x \mathbf{1}[y = g_x] + (1 - \lambda_x)\mathrm{Unif}(\mathcal{Y})] + \varepsilon\,\mathrm{Unif}(\mathcal{H}(x))$ with $\varepsilon = 0.03$. Competence levels $\lambda_x$ are an evenly spaced grid on $[0, 0.9]$ assigned by a fixed permutation with a seed independent of the block structure, so difficulty and universality are uncorrelated by construction. Because the loss is against an exact target rather than samples, the resulting base distribution is matched across universality conditions without any per-condition calibration.

**Tabular policy.** Free parameters $w_1 \in \mathbb{R}^{N\times 10}$ and $w_2 \in \mathbb{R}^{N\times 10\times 10}$ with $\pi(y \mid x) = \mathrm{softmax}(w_1[x])_{y_1} \cdot \mathrm{softmax}(w_2[x, y_1])_{y_2}$. This factorisation is complete: 10 free first-token logits plus $10 \times 10$ free second-token logits parameterise the entire 100-dimensional simplex for each prompt. Given the base transformer's exact log-probabilities we set $w_1 = \log \sum_{y_2} \pi(y \mid x)$ and $w_2 = \log \pi(y \mid x) - w_1$, which reproduces the base distribution to within $10^{-4}$ in max-norm (asserted at run time).

**RL.** The default objective is the exact-expectation policy gradient with the exact mean baseline, $\nabla_\theta J = \frac{1}{N} \sum_x \sum_y \pi_\theta(y \mid x)\,[r(x, y) - V(x)]\,\nabla_\theta \log \pi_\theta(y \mid x)$, implemented as a weighted cross-entropy with detached weights. This is the infinite-group limit of RLOO / Dr. GRPO and removes sampling noise from the comparison; the sampled-GRPO experiment uses group size 8, group-standard-deviation normalisation and one inner epoch. Optimiser Adam (default) or SGD, 400 steps, learning-rate grid $\{3 \times 10^{-4}, 10^{-3}, 3 \times 10^{-3}, 10^{-2}, 3 \times 10^{-2}\}$ for Adam and $\{0.3, 1, 3\}$ for SGD, applied identically to both policy classes. The reported operating point is, for each policy class and cell, the learning rate maximising the final verifier reward, which is what a practitioner without ground truth would select.

**Measurement.** $\rho$, $\Lambda$ and true accuracy are computed from the exact enumerated distribution $\pi_\theta(\cdot \mid x)$ over all 100 responses at all $N$ prompts, at every logged step; they are not Monte-Carlo estimates, and the only stochasticity is over seeds (transformer initialisation, leak-string draw, and rollout sampling in the GRPO condition). Two probes are exceptions: the gradient decomposition and the coherence measurement average over a fixed random subsample of 24 and 12 prompts respectively, for cost.

Per-prompt log-odds are clipped to $\pm 20$ before averaging, so a prompt whose leak mass has collapsed cannot dominate $\Lambda$. At the reported operating points this binds on a median of $1.6\%$ of prompts (worst case $18.8\%$) and it does not change the ordering of any reported series. The final metric of every run is recomputed in float64 from stored per-prompt masses, so the $10^{-8}$ floor used during online logging does not enter any reported number.

The two null-model checks in Section 5 agree to the last bit for a reason: 400 steps of mirror descent at $\eta = 0.05$ reach exactly $\pi_0 e^{20r}$, which is the $\beta = 0.05$ endpoint of the KL grid. They are one computation checked two ways, not two independent checks.

**Per-experiment protocol.** Seeds, learning-rate grids and the operating-point rule vary between experiments; Table 1 records what was actually run.

| | sweep | seeds | learning rates | notes |
|---|---|---|---|---|
| E1 | universality, contiguous blocks | 5 | 5, shared by both arms | headline sweep |
| E2 | universality, permuted blocks | 3 | 4, shared by both arms | base matched to $10^{-3}$ |
| E3 | universality $\times$ error rate | 3 | 1 pair ($10^{-3}$ / $3\times10^{-2}$) | clean SFT floor |
| E4 | sampled GRPO | 3 | 3, shared by both arms | $G = 8$, std normalisation |
| E5 | KL sweep | 3 | 1 ($10^{-3}$) | 6 values of $\beta$ |
| E6 | verifier hardening | 3 | 2 | strict argmax on verifier reward |
| E7 | prompt-set size $N \in \{16, 64\}$ | 3 | 3, shared by both arms | tests $u$ versus $m$ |
| E8 | plain SGD | 5 | 3, shared by both arms | robustness |

Table 1: What was run in each experiment. Where a grid is shared by both arms the operating point is the smallest learning rate within $10^{-3}$ of the best verifier reward, chosen independently for each arm.

# C Additional results

# D Reproducibility

All code, raw result files and figure scripts are included with the submission. A single machine with two CPU cores reproduces every experiment in the paper; no GPU is used. `src/env.py` defines the task and verifier, `src/policies.py` the two policy classes, `src/rlvr.py` the objectives and exact measurement, `src/theory.py` the numerical verification of Proposition 1 and Lemma 1,

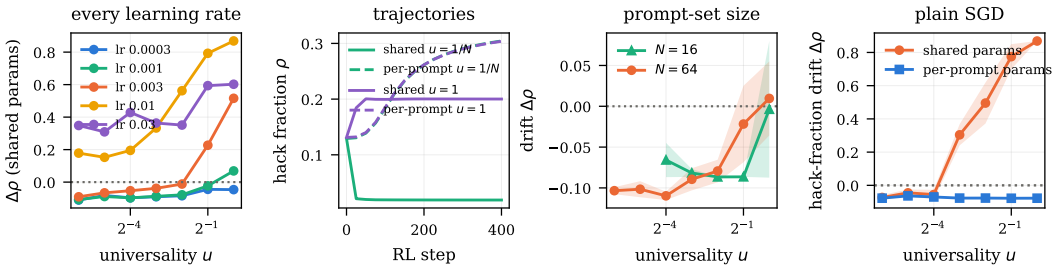



Figure 5: **Left:** the universality effect across the whole Adam learning-rate grid, rather than at the reward-selected operating point. **Centre left:** hack-fraction trajectories for the extreme universality conditions, both policy classes. **Centre right:** the sweep repeated at two prompt-set sizes, plotted against $u$. **Right:** the same universality sweep under plain SGD.

and `src/exp*.py` the experiments. `src/figures.py` regenerates every figure and the macro file holding every measured number quoted in the text; the only hand-entered constants are the step count, the contamination rate and the GRPO group size, which are settings rather than results.